Explainer Studio Pro — Complete Technical Documentation

A technical audit of the project exactly as it exists in the source tree, written so another AI developer can maintain, debug and extend it without rebuilding it.

0. Scope, method and verification status

- **Audited:** every file in the project (25 files, 1,060 lines: 4 root files and 21 files in `src/`). References such as `LiveExplainerPage.jsx:105` come from numbered listings taken at audit time. Each important reference also names a function or code anchor, so it can still be found if line numbers drift.
- **Not modified:** nothing in the project was changed and no ZIP was produced.
- **Build check:** a copy of the project was installed fresh in a temporary folder and built with `vite build`. It succeeded (Vite 5.4.21, 59 modules transformed).
- **NOT verified:** the app was never run in a browser during this audit. The FFmpeg export, Pixabay calls, clip downloads, audio sync and drag interactions are untested at runtime. Read this as: the project compiles, runtime behaviour is unconfirmed.
- **Copy differences:** this audits the folder generated in this chat. If your local copy was merged or edited by hand, your source files win. Paste the real file before any change.

Headline findings

1. **There is no project system.** No project creation, saving, loading or management exists. Every visit to the Live Explainer starts a new session from `DEFAULT_SCRIPT`. Only API keys and caption style persist (in `localStorage`).
2. **There is no router.** Navigation is one `useState` in `App.jsx`. Refresh returns to the home page and the browser Back button does nothing useful.
3. **The OpenRouter key is stored but no code uses it.**
4. **Scenes cannot be deleted or reordered.** They can only be added, edited (text, duration) and have their clip swapped.
5. **The Animation Explainer page is an intentionally empty placeholder.**
6. **No tests, no lockfile, no lint config, no `.env`, no static assets folder.**

Part 1. Complete file inventory

1.1 Directory tree

```
explainer-studio/
  README.md
  index.html
  package.json
  vite.config.js
  src/
    main.jsx
    App.jsx
    pages/
      HomePage.jsx
      LiveExplainerPage.jsx
      AnimationExplainerPage.jsx
    components/live/
      StudioHeader.jsx  Sidebar.jsx  ScriptTab.jsx  ScenesTab.jsx
      PixabayTab.jsx  AudioTab.jsx  CaptionSettings.jsx
      PreviewPlayer.jsx  Timeline.jsx  SettingsModal.jsx  ExportOverlay.jsx
    hooks/
      usePlayback.js  useVideoPool.js
    utils/
      helpers.js  captionStyle.js  exportVideo.js
```

There is **no** `public/`, `assets/`, `styles/` or `tests/` folder, no `.css` file (styling is Tailwind utility classes plus a small `<style>` block in `index.html`), no lockfile and no environment file. No file is currently unused: every file below is imported by another file or by the build.

1.2 Root files

File (all in project root)	What it does	Depends on	Used by	Class
`package.json` (17 lines)	npm manifest. Name `explainer-studio`, `type: module`. Scripts: `dev`, `build`, `preview`. Dependencies: `@ffmpeg/ffmpeg`, `@ffmpeg/util`, `react`, `react-dom`. Dev: `@vitejs/plugin-react`, `vite`.	npm registry	npm, Vite	Configuration, essential
`vite.config.js` (8 lines)	Vite config. Enables `@vitejs/plugin-react` and sets `optimizeDeps.exclude` to the two `@ffmpeg` packages so Vite does not pre-bundle the FFmpeg worker.	`vite`, `@vitejs/plugin-react`	Vite	Configuration, essential
`index.html` (29 lines)	App shell. Loads Tailwind from CDN and sets `tailwind.config` (`darkMode: class`, a `brand` colour palette). Loads Font Awesome 6.4.0 (cdnjs) and Inter (Google Fonts). Inline CSS for scrollbars and `.timeline-block`. Contains `<div id="root">` and the module script `/src/main.jsx`.	Tailwind CDN, cdnjs, Google Fonts	Vite (entry)	Configuration, essential
`README.md` (7 lines)	Install and run commands plus a note about CDN Tailwind and ffmpeg.wasm.	none	humans	Optional documentation

1.3 Source files, entry and pages

File	What it does and key exports	Depends on	Used by	Controls	Class
`src/main.jsx` (5)	Mounts React: `createRoot(document.getElementById('root')).render(<App />)`. No `StrictMode`.	`react`, `react-dom/client`, `./App`	`index.html`	App startup	Essential

File	What it does and key exports	Depends on	Used by	Controls	Class
`src/App.jsx` (13)	`App()` holds `view` state (`'home'`, `'live'` or `'animation'`, `:7`) and `back` (`:8`). Renders one of the three pages.	`HomePage`, `LiveExplainerPage`, `AnimationExplainerPage`	`main.jsx`	Navigation	Essential
`src/pages/HomePage.jsx` (52)	`OPTIONS` array (`:1`) defines the two cards. `HomePage({ onSelect })` (`:20`). Card `onClick={() => onSelect(o.id)}` (`:33`). The animation card shows a Coming soon badge.	none	`App.jsx`	Start screen	Essential
`src/pages/AnimationExplainerPage.jsx` (13)	`AnimationExplainerPage({ onBack })`. Header with a Home button and an empty `<main>`.	none	`App.jsx`	Placeholder	Placeholder
`src/pages/LiveExplainerPage.jsx` (151)	The studio container. Owns all studio state, script parsing, auto-match, scene edits, audio and video sync, and export launching. Exports `LiveExplainerPage({ onBack })` (`:26`). Local helper `loadAudio` (`:19`).	`usePlayback`, `useVideoPool`, `helpers`, `exportVideo`, `captionStyle`, and 9 components	`App.jsx`	The whole Live Explainer	Essential

1.4 Source files, components (`src/components/live/`)

File	What it does and key exports	Depends on	Used by	Controls	Class
`StudioHeader.jsx` (28)	Top bar: back button, title, API Keys button, Render Export button (disabled while exporting). `StudioHeader({ onBack, onSettings, onExport, exporting })`.	none	`LiveExplainerPage`	Header actions	Essential
`Sidebar.jsx` (23)	Tab strip and scroll container. `TABS` (`:1-7`) lists `script`, `scenes`, `pixabay`, `audio`, `captions`. `Sidebar({ tab, onTab, children })` (`:9`).	none	`LiveExplainerPage`	Sidebar navigation	Essential
`ScriptTab.jsx` (17)	Script textarea and the Parse & Auto-Match button. `ScriptTab({ script, onChange, onParse })`.	none	`LiveExplainerPage`	Script input	Essential

File	What it does and key exports	Depends on	Used by	Controls	Class
`ScenesTab.jsx` (35)	Scene cards: duration input, caption textarea, Search Swap, Add Scene. `ScenesTab({ scenes, activeId, onChange, onAdd, onSwap })`.	none	`LiveExplainerPage`	Scene editing	Essential
`PixabayTab.jsx` (70)	Pixabay video search and Apply. Contains `search` (`:10-21`). `PixabayTab({ apiKey, seed, sceneNumber, onApply })`.	`react`	`LiveExplainerPage`	Clip search and swap	Essential
`AudioTab.jsx` (54)	Voiceover upload and player, Sync button, background music upload, volume slider. `AudioTab({ voice, bg, bgVol, onVoice, onBg, onBgVol, onSync })`.	none	`LiveExplainerPage`	Audio inputs	Essential
`CaptionSettings.jsx` (85)	Caption style panel (presets, font, size, B/I/AA, colours, background style, position, reset). Local `Seg` (`:3`) and `Row` (`:12`). `CaptionSettings({ value, onChange })` (`:16`).	`captionStyle`	`LiveExplainerPage`	Caption styling UI	Essential
`PreviewPlayer.jsx` (51)	Canvas preview, aspect buttons, caption overlay. Own `requestAnimationFrame` paint loop (`useEffect` at `:14`). `PreviewPlayer({ active, pool, aspect, onAspect, caption })` (`:7`).	`helpers` (`ASPECTS`), `captionStyle` (`previewCaptionStyle`)	`LiveExplainerPage`	Visual preview	Essential
`Timeline.jsx` (64)	Transport buttons, time readout, scene blocks (drag to resize), draggable playhead, range scrubber. Handlers `drag` (`:8`), `startResize` (`:13`), `seekX` (`:18`), `dragHead` (`:22`).	`helpers` (`formatTime`)	`LiveExplainerPage`	Timing and seeking	Essential

File	What it does and key exports	Depends on	Used by	Controls	Class
`SettingsModal.jsx` (29)	API key modal with local draft state. `SettingsModal({ keys, onSave, onClose })`.	`react`	`LiveExplainerPage`	API keys	Essential
`ExportOverlay.jsx` (14)	Full-screen progress card. `ExportOverlay({ progress, label })`. No cancel button.	none	`LiveExplainerPage`	Export progress display	Essential

1.5 Source files, hooks and utilities

File	What it does and key exports	Depends on	Used by	Controls	Class
`src/hooks/usePlayback.js` (31)	Default export `usePlayback(total)`. Owns `time`, `playing`, an rAF loop and refs. Returns `{ time, playing, setPlaying, toggle, stop, seek }`.	`react`	`LiveExplainerPage`	Playhead clock	Essential
`src/hooks/useVideoPool.js` (28)	Default export `useVideoPool(scenes)`. Keeps one hidden `<video>` per scene id in a ref. Returns the ref (`pool.current[id] = { video, url }`).	`react`, `helpers` (`getClipBlob`)	`LiveExplainerPage`	Per-scene video elements	Essential
`src/utils/helpers.js` (71)	`FALLBACK_VIDEOS` (`:1`), `DEFAULT_SCRIPT`, `ASPECTS` (`:22-26`), `formatTime` (`:28`), `extractKeywords` (`:33-36`), `makeScene` (`:38-44`), `withStarts` (`:46-49`), `sceneIndexAt` (`:51-54`), `getClipBlob` (`:59-64`), `downloadBlob` (`:66-71`).	none	page, hooks, Timeline, PreviewPlayer, `exportVideo`	Scene model, timing, clip cache	Essential
`src/utils/captionStyle.js` (49)	`FONTS` (`:1`), `DEFAULT_CAPTION` (`:9`), `PRESETS` (`:23`), `captionFontSize` (`:30`), `fontString` (`:31`), `hexToRgba` (`:32`), `previewCaptionStyle` (`:38`).	none	`CaptionSettings`, `PreviewPlayer`, `exportVideo`, page	Caption look (shared by preview and export)	Essential
`src/utils/exportVideo.js` (116)	`loadFFmpeg` (`:9`), `captionPng` (`:21`), `exportVideo(...)` (`:55`). Builds one FFmpeg command and returns `{ blob, missing }`.	`@ffmpeg/ffmpeg`, `@ffmpeg/util`, `helpers`, `captionStyle`	`LiveExplainerPage.doExport`	MP4 rendering	Essential

Part 2. Application architecture

2.1 Feature status at a glance

Feature	Status	Files
Entry point	Implemented	`index.html`, `main.jsx`
Routing and navigation	Implemented as a state switch only (no URLs)	`App.jsx:7`
Home page	Implemented	`HomePage.jsx`
Project creation, saving, loading, management	**NOT IMPLEMENTED**	none
Script editor	Implemented (plain textarea)	`ScriptTab.jsx`
Script parsing and scene generation	Implemented	`LiveExplainerPage.jsx:67`, `helpers.js:38`
Automatic media matching	Implemented, needs a Pixabay key	`LiveExplainerPage.jsx:55`
Pixabay integration	Video search implemented. Audio search is only a notice message.	`PixabayTab.jsx`
Video preview	Implemented	`PreviewPlayer.jsx`
Timeline	Implemented (resize, seek). No reorder or delete.	`Timeline.jsx`
Scene editing	Text, duration, add, clip swap. No delete or reorder.	`ScenesTab.jsx`
Voiceover	Implemented, one global file	`AudioTab.jsx`, page `:19`, `:76`

Feature	Status	Files
Background music	Implemented (upload, loop, volume). No trimming or fade.	`AudioTab.jsx`
Audio synchronization	Implemented	`LiveExplainerPage.jsx:82-102`
Rendering and export	Implemented with FFmpeg, untested at runtime	`exportVideo.js`
Export progress and errors	Progress bar, alert on failure, no cancel	`ExportOverlay.jsx`, page `:105-115`
Settings and API keys	Implemented. OpenRouter key unused.	`SettingsModal.jsx`, page `:44-47`
Caption styling	Implemented, saved in `localStorage`	`CaptionSettings.jsx`, `captionStyle.js`
Data storage	`localStorage` for 3 keys only	page `:40-47`, `:146`

2.2 Sequence of events: from opening the app to a finished video

1. **Startup.** The browser loads `index.html`, which pulls Tailwind, Font Awesome and Inter from CDNs, then runs `/src/main.jsx`. `main.jsx` renders `<App />`.
2. **Home.** `App` starts with `view = 'home'` and renders `HomePage`. Clicking a card calls `onSelect('live')` or `onSelect('animation')`, which is `setView`.
3. **Studio mounts.** `LiveExplainerPage` initialises its state. `keys` are read from `localStorage` (`pixabay_key`, `openrouter_key`, `:44-47`). `caption` is read from `localStorage` (`caption_style`, `:40-42`) and merged over `DEFAULT_CAPTION`.
4. **First parse.** A mount effect calls `parse()` (`:71`). It splits `DEFAULT_SCRIPT` into scenes, each created by `makeScene`, stores them with `setRaw`, stops playback, and starts `autoMatch(list)`.
5. **Scene model.** `scenes = useMemo(withStarts(raw))` (`:28`) adds `startTime` to each scene. `total` (`:29`) is the sum of durations.
6. **Video loading.** `useVideoPool(scenes)` creates a hidden `<video>` per scene. Each clip is downloaded as a Blob by `getClipBlob` and played from a local object URL. If the download fails, the video falls back to the direct URL.

7. **Auto-match.** If a Pixabay key exists, `autoMatch` searches each scene's `keywords` one after another and replaces `videoUrl` with the first hit, which triggers the pool to load the new clip.
8. **Editing.** The user edits through the sidebar tabs. All edits go through `setRaw` (`updateScene`, `addScene`, `syncToVoice`) or the other `useState` setters in the page.
9. **Preview.** `usePlayback(total)` advances `time` on each animation frame. The sync effect (`:88-102`) keeps audio and the active scene's video aligned with `time`. `PreviewPlayer` paints the active video to a canvas and overlays the caption.
10. **Export.** `Render Export` calls `doExport` (`:105`). It calls `exportVideo(...)`, which loads FFmpeg, writes every clip, caption PNG and audio file into FFmpeg's in-memory file system, runs one `ff.exec`, reads `out.mp4`, and returns a Blob. `downloadBlob` saves it as `explainer-studio-export.mp4`.

2.3 Feature-by-feature detail

Routing and navigation. `App.jsx` only. `view` decides which page renders. `onBack` / `back` returns to `'home'`. No URL, history or deep links.

Script parsing. `parse` (`LiveExplainerPage.jsx:67`) splits `script` on line breaks, trims, drops empty lines, and maps each line through `makeScene`. Every parse replaces all scenes, discarding earlier edits and applied clips.

Scene generation. `makeScene(text, i)` (`helpers.js:38`) returns `{ id: crypto.randomUUID(), text, duration: 5, videoUrl: FALLBACK_VIDEOS[i % 5], keywords: extractKeywords(text) }`.

Keyword extraction. `extractKeywords` (`helpers.js:33`) lowercases, strips punctuation, removes words in the `STOP` set (`:31`) and words of 3 letters or fewer, keeps the first two remaining words, and falls back to `'technology'`.

Automatic media matching. `autoMatch` (`:55`) returns immediately if there is no key. Otherwise, for each scene in order it calls `https://pixabay.com/api/videos/?key=...&q=<keywords>&per_page=3`, takes `hits[0].videos.small.url` (or `tiny`), and updates only that scene by id. Failures are silently ignored.

Pixabay tab. `search` (`PixabayTab.jsx:10-21`) handles three cases: audio type shows a notice, missing key shows an error status, otherwise it fetches 10 video hits. Apply calls `onApply(url)`, which the page maps to `updateScene(active.id, { videoUrl: url })` (`:129`). The tab can be triggered from the Scenes tab: Search Swap sets `pixSeed` (`:125`) and the tab's effect on `seed.n` (`:23`) fills the query and searches.

Video preview. `PreviewPlayer` runs its own animation-frame loop (`:14-30`). It reads the latest `active` scene and `pool` from a ref, clears the canvas to black, and draws the active video with cover-fit scaling. The canvas size comes from `ASPECTS[aspect].preview`. The caption is a DOM `<span>` styled by `previewCaptionStyle`, shown only when `caption.show`.

Timeline. `Timeline.jsx` renders blocks sized by `duration / total`. Pressing on a block starts `startResize`, which changes that scene's duration (minimum 1.5 s, rounded to 0.1). Click on a block seeks to its `startTime`. The playhead (`dragHead`) and the range input both call `onSeek`. Dragging does **not** reorder scenes.

Voiceover and music. `loadAudio` (`:19`) creates an object URL and an `Audio` to read `duration`, then stores `{ url, file, name, duration }`. Passing `null` clears. `syncToVoice` (`:76`) sets every scene's duration to `voice.duration / scenes.length`.

Settings and API keys. `SettingsModal` edits a local draft. Save calls the page's `onSave` (`:146`), which updates `keys` and writes `pixabay_key` and `openrouter_key` to `localStorage`.

Part 3. The brain: how the app actually works

3.1 State (all in `LiveExplainerPage`, `:26-52`)

State / ref	Shape	Set by	Read by
`raw` (`:27`)	array of scenes without start times	`parse`, `autoMatch`, `updateScene`, `addScene`, `syncToVoice`	`scenes`, `syncToVoice`
`scenes` (`:28`, derived)	`raw` plus `startTime`	`useMemo` over `withStarts`	almost everything
`total` (`:29`, derived)	seconds	`useMemo` sum	`usePlayback`, Timeline, export
`script` (`:31`)	string	`ScriptTab`	`parse`
`tab` (`:32`)	`'script'`, `'scenes'`, `'pixabay'`, `'audio'` or `'captions'`	`Sidebar`, Search Swap	conditional renders `:122-135`

State / ref	Shape	Set by	Read by
`aspect` (`:33`)	`'16:9'`, `'9:16'` or `'1:1'`	`PreviewPlayer` buttons	preview, export
`voice` (`:34`), `bg` (`:35`)	`{ url, file, name, duration }` or `null`	`loadAudio`	AudioTab, sync effect, export
`bgVol` (`:36`)	number 0 to 1 (default 0.2)	AudioTab slider	audio element volume, export
`pixSeed` (`:37`)	`{ q, n }`	`onSwap`	`PixabayTab` effect
`showSettings` (`:38`)	boolean	header button	modal render
`exp` (`:39`)	`{ p, label }` or `null`	`doExport`	`ExportOverlay`, header disabled state
`caption` (`:40`)	see `DEFAULT_CAPTION`	`CaptionSettings`	preview, export, `localStorage`
`keys` (`:44`)	`{ pixabay, openrouter }`	`SettingsModal`	`autoMatch`, `PixabayTab`
`voiceEl`, `bgEl` (`:83`)	refs to `Audio` objects	effects `:84-85`	sync effect
`pb` (`:49`)	`{ time, playing, setPlaying, toggle, stop, seek }`	`usePlayback`	page, Timeline
`pool` (`:50`)	ref of `{ id: { video, url } }`	`useVideoPool`	sync effect, PreviewPlayer
`active`, `activeIndex` (`:51-52`)	scene at `pb.time`	`sceneIndexAt`	Scenes highlight, Pixabay Apply, preview

3.2 How components communicate

There is **no context, store or router**. All state lives in `LiveExplainerPage` and flows down as props. Children send changes up through callback props (`onChange`, `onAdd`, `onApply`, `onSeek`, `onResize`, and so on). The only direct component-to-component channel is

`pool`, a ref passed to `PreviewPlayer`. The only cross-component trigger is `pixSeed`, a counter object that lets Scenes tab start a search in the Pixabay tab.

3.3 Durations and start times

- `duration` is stored per scene. `startTime` is never stored in `raw`.
- `withStarts` (`helpers.js:46`) walks the list adding durations: scene 0 starts at 0, scene n starts at the sum of earlier durations. `total` is the sum of all durations.
- Duration writers: default 5 (`makeScene`), ScenesTab input (`ScenesTab.jsx:17`, minimum 1, falls back to 3 on invalid input), Timeline resize (minimum 1.5), `syncToVoice` (equal split rounded to 2 decimals).
- `sceneIndexAt(scenes, t)` returns the scene where `startTime <= t < startTime + duration`, or the last scene if none.

3.4 Video clips

`useVideoPool` (`useVideoPool.js:8-25`) runs when `scenes` changes. It deletes entries for removed scene ids, and for each scene whose `videoUrl` differs from the stored `url` it creates a `<video>` (muted, loop, `crossOrigin='anonymous'`). `getClipBlob(url)` (`helpers.js:59`) caches a promise that resolves to a Blob or `null`. If it resolves, the video plays from `URL.createObjectURL(blob)`. If it returns `null`, the video uses the direct URL. If the video errors while still in CORS mode, `onerror` (`:18`) removes `crossorigin` and retries the direct URL.

3.5 Voiceover and music synchronisation

The sync effect (`LiveExplainerPage.jsx:88-102`) runs whenever `pb.time`, `pb.playing`, `voice`, `bg` or `active.id` changes, which is every frame during playback.

```
for each audio element (voice, bg):
  target = (looping and has duration) ? time % duration : time
  if |audio.currentTime - target| > 0.25 -> set audio.currentTime = target
  if playing and audio.paused -> play()
  if not playing and not audio.paused -> pause()
for each video in pool:
  if not the active scene -> pause
  else target = (time - active.startTime) % video.duration
       if paused or |currentTime - target| > 0.3 -> set currentTime = target
       play or pause to match pb.playing
```

The AudioTab also shows a native `<audio controls>` for the voiceover. It plays independently and is not synchronised with the timeline.

3.6 Playback clock

`usePlayback` (`usePlayback.js`) keeps `time` in state and the running value in `tRef`. While `playing`, an animation-frame loop adds elapsed seconds. At the end of the timeline (`:15`) it resets to 0 and stops. `seek` clamps to `0..total`. `stop` pauses and seeks to 0. The Restart button calls `pb.seek(0)` and does not change play state.

3.7 Rendering engine and final video

All rendering happens in the browser through `exportVideo` (`exportVideo.js:55-116`). Execution flow:

```
doExport (page :105)
  -> guard: total must be > 0
  -> pb.setPlaying(false); setExp({p:0,label})
  -> exportVideo({scenes, aspect, voice, bg, bgVol, total, caption,
onProgress})
       loadFFmpeg() : single-thread core 0.12.6 from unpkg, cached in module
variable
       attach 'log' (keep last 6 lines) and 'progress' listeners
       for each scene i:
         blob = await getClipBlob(videoUrl)
         blob ? write clip{i}.mp4 and input '-stream_loop -1 -t dur'
              : black 'lavfi' colour input, missing++
         write cap{i}.png (captionPng) as the next input
         filter: scale+crop to W:H, 30 fps, overlay caption
       concat all scene chains, format yuv420p -> [vout]
       optional voice.<ext>, bg.<ext> (bg uses -stream_loop -1)
       audio filter: voice only, bg only, or amix of both then volume=2
       ff.exec([... libx264 ultrafast crf 27, aac 128k, faststart, -t total,
out.mp4])
       exit code != 0 -> throw Error with last log lines
       read out.mp4 -> Blob
       finally: remove listeners, delete all temp files
  -> downloadBlob(blob, 'explainer-studio-export.mp4')
  -> alert if missing > 0
  -> finally setExp(null)
```

Output sizes come from `ASPECTS[aspect].out` (854x480, 480x854, 576x576). Progress is 0 while loading FFmpeg, 0.01 to 0.15 while preparing scenes, then 0.15 to 1.0 from FFmpeg's `progress` event.

3.8 Caption pipeline

`caption` is one object shared by both renderers. The preview uses `previewCaptionStyle(caption, cw, ch)` to style a DOM span. Export uses `captionPng` to draw the same style on a transparent canvas and FFmpeg overlays that PNG. Both compute the font size as `min(width, height) * 0.06 * size / 100` through `captionFontSize`, so proportions match, but line wrapping can differ slightly (preview limits width with CSS at 82 percent, export wraps in code).

3.9 Persistence

See Part 5. Only `pixabay_key`, `openrouter_key` and `caption_style` are persisted.

3.10 Error handling as implemented

Place	Behaviour
`autoMatch` (`:63`)	Empty `catch`. Failures are silent and the fallback clip stays.
`PixabayTab.search`	Shows `Search failed.` or a status message.
`getClipBlob`	`.catch(() => null)`. Resolves to `null` on any failure.
`useVideoPool`	`onerror` retry without CORS.
Audio and video `play()`	`.catch(() => {})`, ignored.
`caption` initial read (`:40-42`)	`try/catch`, falls back to defaults.
`doExport` (`:112-114`)	`alert('Export failed: ...')`, `console.error`, overlay cleared in `finally`.
`exportVideo` cleanup	`finally` removes listeners and deletes temp files.
`localStorage.setItem` (`:43`, `:146`)	Not wrapped in `try/catch`.
Whole app	No React error boundary.

3.11 External services

Service	Where	Purpose
Pixabay videos API `https://pixabay.com/api/videos/`	`LiveExplainerPage.jsx:59`, `PixabayTab.jsx:17`	Clip search (needs user key)
unpkg `@ffmpeg/core@0.12.6`	`exportVideo.js:12`	FFmpeg wasm core, fetched at first export
Mixkit preview clips	`helpers.js` `FALLBACK_VIDEOS`	Default clips when no key
Tailwind CDN, cdnjs Font Awesome, Google Fonts	`index.html`	Styling and icons
OpenRouter	nowhere	Key is stored, never called